

Assignment Report

Multithreading in C using POSIX Threads

ARITRA TALUKDAR;CSB24034

Contents

1	Aim	2
2	Theory	2
2.1	What is a Thread?	2
2.2	POSIX Threads (pthread)	2
2.3	Race Condition	3
2.4	Mutex	3
3	Program Without Synchronization	3
3.1	Source Code	3
4	Explanation of the Program	4
4.1	Global Counter	4
4.2	Thread Function	4
4.3	Thread Creation	4
4.4	Thread Joining	5
5	Observed Output	5
6	Program Using Mutex Synchronization	5
6.1	Source Code	5
7	Explanation of Mutex Functions	7
7.1	Mutex Declaration	7
7.2	Mutex Initialization	7
7.3	Locking the Mutex	7
7.4	Unlocking the Mutex	7
7.5	Destroying the Mutex	7
8	Correct Output	8
9	Learning Outcomes	8
10	Conclusion	8

1 Aim

The aim of this assignment is to understand multithreading in C using POSIX Threads (pthread). The program demonstrates:

- Creation of multiple threads
- Shared global variable access
- Race condition in multithreading
- Synchronization using mutex
- Proper thread completion

2 Theory

2.1 What is a Thread?

A thread is the smallest unit of execution inside a process. Multiple threads can run concurrently within the same program.

Threads share:

- Global variables
- Heap memory
- Program code

Each thread has its own:

- Stack
- Program counter
- Registers

2.2 POSIX Threads (pthread)

POSIX Threads is a standard API used for multithreading in C and C++ programs on Linux and Unix systems.

The library is included using:

```
1 #include <pthread.h>
```

2.3 Race Condition

A race condition occurs when multiple threads access and modify shared data simultaneously without synchronization.

In this assignment, all threads increment the same global variable:

```
1 counter++;
```

This operation is internally divided into:

1. Read value
2. Increment value
3. Write value back

If multiple threads perform these steps simultaneously, updates may be lost, producing incorrect output.

2.4 Mutex

A mutex (Mutual Exclusion) is a synchronization mechanism used to protect critical sections.

Only one thread can lock the mutex at a time.

Functions used:

- `pthread_mutex_lock()`
- `pthread_mutex_unlock()`

This ensures safe access to shared resources.

3 Program Without Synchronization

3.1 Source Code

```
1 //Program without synchronization
2 #include<stdio.h>
3 #include<pthread.h>
4 #define NUM_THREADS 4
5 #define INCREMENTS 1000000
6 long long counter = 0;
7
8 void* increment_counter(void* arg) {
9     for (int i = 0; i < INCREMENTS; i++) {
10         counter++; // Incrementing the shared counter(UNSAFE)
11     }
12     return NULL;
13 }
14 int main() {
15     pthread_t threads[NUM_THREADS];
16 }
```

```

17 // Create multiple threads to increment the counter
18 for (int i = 0; i < NUM_THREADS; i++) {
19     pthread_create(&threads[i], NULL, increment_counter, NULL
20 );
21 }
22
23 // Wait for all threads to finish
24 for (int i = 0; i < NUM_THREADS; i++) {
25     pthread_join(threads[i], NULL);
26 }
27
28 // Print the final value of the counter
29 printf("Final counter value: %lld\n", counter);
30 printf("Expected counter value: %lld\n", (long long)
31 NUM_THREADS * INCREMENTS);
32 return 0;
33 }

```

4 Explanation of the Program

4.1 Global Counter

```
1 long long counter = 0;
```

This variable is shared among all threads.

4.2 Thread Function

```
1 void* increment_counter(void* arg)
```

This function is executed by every thread.

4.3 Thread Creation

```

1 pthread_create(&threads[i],
2               NULL,
3               increment_counter,
4               NULL);

```

Creates a new thread.

Parameters:

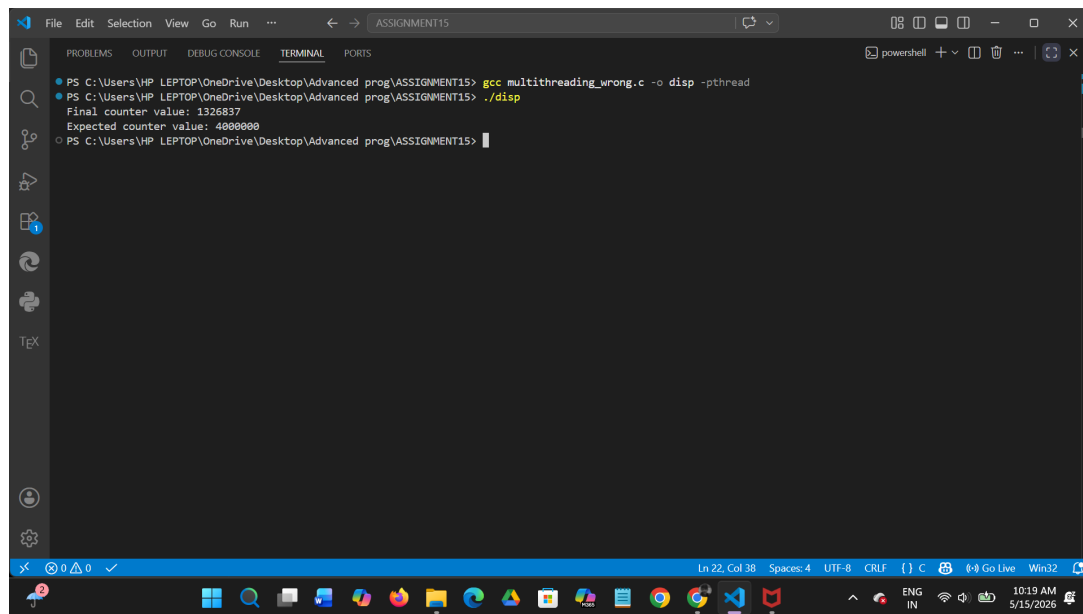
1. Thread ID
2. Thread attributes
3. Function executed by thread
4. Argument passed to function

4.4 Thread Joining

```
1 pthread_join(threads[i], NULL);
```

Waits for the thread to complete execution.

5 Observed Output



```
File Edit Selection View Go Run ... ASSIGNMENT15
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\HP LEPTOP\OneDrive\Desktop\Advanced prog\ASSIGNMENT15> gcc multithreading_wrong.c -o disp -pthread
PS C:\Users\HP LEPTOP\OneDrive\Desktop\Advanced prog\ASSIGNMENT15> ./disp
Final counter value: 1326837
Expected counter value: 4000000
PS C:\Users\HP LEPTOP\OneDrive\Desktop\Advanced prog\ASSIGNMENT15>
```

The output is incorrect because multiple threads access the shared counter simultaneously, causing a race condition.

6 Program Using Mutex Synchronization

6.1 Source Code

```
1 //Modified correct program using mutex to protect the shared
  counter variable
2 #include<stdio.h>
3 #include<pthread.h>
4 #define NUM_THREADS 4
5 #define INCREMENTS 1000000
6 long long counter = 0;
7 // Mutex variable
8 pthread_mutex_t lock;
9
10 /*
11 Thread function
12 */
13 void* increment_counter(void* arg)
14 {
```

```

15     for(int i = 0; i < INCREMENTS; i++)
16     {
17         // Lock critical section
18         pthread_mutex_lock(&lock);
19
20         counter++;
21
22         // Unlock critical section
23         pthread_mutex_unlock(&lock);
24     }
25
26     return NULL;
27 }
28
29 int main()
30 {
31     pthread_t  threads[NUM_THREADS];
32
33     /*
34     Initialize mutex
35     */
36     pthread_mutex_init(&lock, NULL);
37
38     /*
39     Create threads
40     */
41     for(int i = 0; i < NUM_THREADS; i++)
42     {
43         pthread_create(&threads[i],
44                       NULL,
45                       increment_counter,
46                       NULL);
47     }
48
49     /*
50     Wait for all threads
51     */
52     for(int i = 0; i < NUM_THREADS; i++)
53     {
54         pthread_join(threads[i], NULL);
55     }
56
57     /*
58     Print final result
59     */
60     printf("Final Counter Value = %lld\n", counter);
61     printf("Expected Counter Value = %lld\n", (long long)
NUM_THREADS * INCREMENTS);
62     /*
63     Destroy mutex
64     */

```

```
65     pthread_mutex_destroy(&lock);  
66  
67     return 0;  
68 }
```

7 Explanation of Mutex Functions

7.1 Mutex Declaration

```
1 pthread_mutex_t lock;
```

Declares a mutex variable.

7.2 Mutex Initialization

```
1 pthread_mutex_init(&lock, NULL);
```

Initializes the mutex before use.

7.3 Locking the Mutex

```
1 pthread_mutex_lock(&lock);
```

Only one thread can enter the critical section at a time.

7.4 Unlocking the Mutex

```
1 pthread_mutex_unlock(&lock);
```

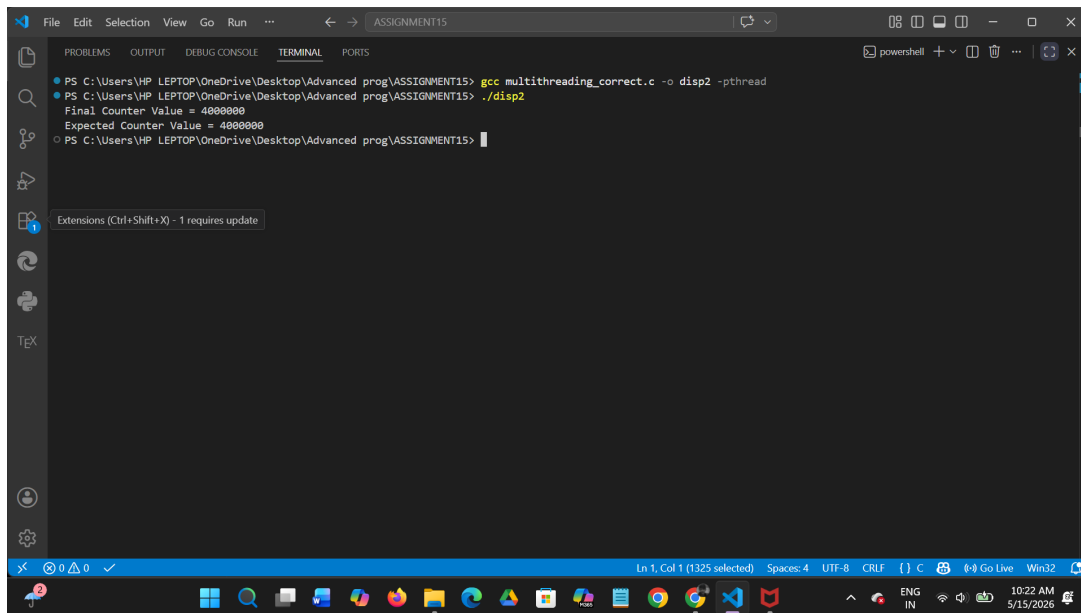
Releases the mutex for other threads.

7.5 Destroying the Mutex

```
1 pthread_mutex_destroy(&lock);
```

Frees mutex resources.

8 Correct Output



```
File Edit Selection View Go Run ... ASSIGNMENT15
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\HP LEPTOP\OneDrive\Desktop\Advanced prog\ASSIGNMENT15> gcc multithreading_correct.c -o disp2 -pthread
PS C:\Users\HP LEPTOP\OneDrive\Desktop\Advanced prog\ASSIGNMENT15> ./disp2
Final Counter Value = 4800000
Expected Counter Value = 4800000
PS C:\Users\HP LEPTOP\OneDrive\Desktop\Advanced prog\ASSIGNMENT15>
Extensions (Ctrl+Shift+X) - 1 requires update
Ln 1, Col 1 (1325 selected) Spaces: 4 UTF-8 CRLF {} C Go Live Win32
10:22 AM 5/15/2026
```

The output is correct because mutex synchronization protects the critical section and prevents race conditions.

9 Learning Outcomes

After completing this assignment, the following concepts were understood:

- Multithreading in C
- POSIX thread library
- Thread creation and joining
- Shared memory access
- Race conditions
- Critical sections
- Mutex synchronization

10 Conclusion

This assignment successfully demonstrated multithreading using POSIX threads in C. The first program showed how race conditions occur when multiple threads access shared data without synchronization. The second program solved the issue using mutex locks, ensuring safe and correct access to the shared counter variable.